Integrated Report: Imitation Learning for Dual-Arm Robotic Manipulation

Student1: 59371063 XU Ziheng
Student2: 59793561 Yang Muhan
Student3: 59636174 LI Tieyun

Abstract

This integrated project addresses imitation learning for dual-arm robotic manipulation through three complementary components. Student 1 builds a reliable simulation environment, designs a high-success scripted expert policy, develops a structured noisy demonstration generation strategy, and establishes a standardized dataset organization framework. Student 2 implements Behavior Cloning (BC) and Dataset Aggregation (DAgger) baselines, conducts empirical comparisons, and analyzes covariate shift and compounding errors to reveal the limitations of traditional imitation methods. Student 3 applies Diffusion Policy with action chunking and regularization techniques to dual-arm lifting tasks, achieving 86% closed-loop success rate and verifying the critical role of action chunk length and overfitting mitigation. Collectively, this work forms a complete research pipeline from data generation to policy learning and validation, providing empirical insights and technical foundations for robust dual-arm robotic manipulation.

Contents

1 Project Context and Work Division

This project studies imitation learning for dual-arm robotic manipulation. The task requires two robotic arms to coordinate their motions to complete a lifting behavior in simulation. Compared with single-arm manipulation, dual-arm manipulation is more difficult because the policy must control both arms at the same time. If one arm approaches too slowly, grasps at the wrong time, or lifts before the other arm is stable, the whole task can fail. Therefore, this project is a useful setting for studying long-horizon imitation learning, covariate shift, and policy robustness.

The project follows a pipeline from data generation to policy learning and evaluation. Student 1 focuses on the simulation environment, scripted expert policy, noisy demonstration collection, and dataset preprocessing. This part provides the foundation for all later learning methods. Student 3 focuses on Diffusion Policy, including action chunking, training, sampling, and regularization for the final generative policy model. My work as Student 2 is located between these two parts. I focus on the classical imitation learning baselines: Behavior Cloning and DAgger. These baselines are important because they provide a direct comparison for the more advanced Diffusion Policy method and help explain why covariate shift is a serious problem in embodied intelligence.

This report mainly presents my individual contribution within the integrated project. I briefly describe the full project background so that the role of my work is clear, but the main content focuses on my own implementation, experiments, results, and analysis. Specifically, my part includes BC implementation, DAgger implementation, noisy-demonstration evaluation, final 50-episode rollout testing, covariate shift analysis, and supplementary diagnostics.

2 Student 2 Role and Contribution

My responsibility in this project is to implement and analyze traditional imitation learning baselines. The main methods are Behavior Cloning (BC) and Dataset Aggregation (DAgger). BC is the simplest supervised imitation learning baseline. It directly learns a mapping from state to action using expert demonstrations. DAgger improves on BC by collecting states visited by the learned policy and asking the expert to label those states [1]. This process helps reduce the distribution mismatch between expert demonstrations and policy rollouts.

My contribution can be summarized in five parts.

First, I implemented a BC training pipeline for the dual-arm task. The policy takes a low-dimensional robot state vector as input and predicts a dual-arm action vector. I used a PyTorch MLP policy, normalized states and actions, split the dataset at the trajectory level, trained the model using MSE loss, and selected the checkpoint based on validation loss.

Second, I implemented a DAgger pipeline. This required more than simply training another model. The DAgger process needs to roll out the current learner policy, collect the states that the learner actually visits, query the scripted expert for labels, save the relabeled data, aggregate the new data with the previous dataset, and retrain the policy over multiple rounds.

Third, I designed the noisy-demonstration comparison. The main evaluation uses noisy ratios 0.0, 0.3, 0.5, and 1.0. These settings cover clean demonstrations, moderate noise, stronger mixed noise, and fully noisy demonstrations. I also evaluated ratio 0.8 as a diagnostic case, but I report it in the appendix because it behaves more like a high-noise failure case than a normal trend point.

Fourth, I performed final closed-loop evaluation. Instead of relying on training loss or short smoke tests, the final results in this report use 50 rollout episodes for each selected checkpoint. The main metrics are success rate, success count, and mean episode steps. This is important because imitation learning policies must be tested in closed-loop rollouts, where their own actions affect future states.

Finally, I analyzed the results using covariate shift and compounding error. The results show that BC can work well on clean demonstrations but fails quickly when demonstrations become noisy. DAgger significantly improves performance under moderate noise, especially at noisy ratio 0.5. However, DAgger is still not fully robust under very high noise. This analysis helps explain the limitations of classical imitation learning and motivates the Diffusion Policy component.

3 Task and Data Background

3.1 Dual-Arm Lifting Task

The experiments in my part use a dual-arm lifting task in robosuite, a modular robot-learning simulation framework built on MuJoCo [2]. The environment contains two Panda robot arms and an object that needs to be lifted. The two arms must coordinate several stages: approach, alignment, grasping, and lifting. These stages are closely connected. A small error in approach may make grasping unstable, and a small timing mismatch between the two arms may cause the final lifting stage to fail.

This task is suitable for evaluating imitation learning because it is long-horizon and sensitive to compounding errors. A learned policy does not only need to predict the correct expert action on expert states. It also needs to remain stable when its own previous actions lead to slightly different states. This is the core reason why covariate shift matters in this project.

3.2 Clean and Noisy Demonstrations

The training data come from the demonstration pipeline built in the group project. Clean demonstrations are generated by a scripted expert policy. Noisy demonstrations are generated by adding controlled perturbations during data collection. These noisy demonstrations are useful because real-world data are rarely perfect. If a method only works on clean demonstrations, it may not be robust enough for practical robotic learning.

In my experiments, I use mixed datasets with different noisy-demonstration ratios. A ratio of 0.0 means the dataset contains only clean demonstrations. A ratio of 1.0 means the dataset contains only noisy demonstrations. Intermediate ratios combine clean and noisy data. This design makes it possible to compare how BC and DAgger react to different levels of demonstration quality.

The main ratios used in the report are:

- 0.0: fully clean demonstrations;
- 0.3: moderate noisy demonstrations;
- 0.5: stronger mixed noisy demonstrations;
- 1.0: fully noisy demonstrations.

The ratio 0.8 case is also tested, but it is treated as a separate high-noise diagnostic case in the appendix.

3.3 State and Action Representation

The BC and DAgger policies use a 50-dimensional state vector and a 14-dimensional action vector. The state vector contains robot and object information needed for the lifting task. The action vector controls both robot arms. Since the input and output are numerical vectors, an MLP is a reasonable baseline policy architecture.

Before training, both states and actions are normalized using training-set statistics. This prevents dimensions with larger numerical scales from dominating the learning process. During rollout evaluation, predicted normalized actions are converted back to the environment action scale.

4 Behavior Cloning Method

Behavior Cloning treats imitation learning as supervised regression. Given expert demonstration data, each trajectory provides state-action pairs (s_t, a_t), where s_t is the robot state at time step t and a_t is the expert action. The policy is trained to predict the expert action from the current state:

$$\hat{a}_t = \pi_\theta(s_t). \tag{1}$$

The training objective is mean squared error between the predicted action and the expert action:

$$\mathcal{L}_{BC} = \frac{1}{N} \sum_{t=1}^{N} \|\pi_\theta(s_t) - a_t\|_2^2 . \tag{2}$$

In my implementation, the BC policy is a PyTorch MLP. The dataset is split at the trajectory level instead of the timestep level. This is important because if timesteps from the same trajectory are placed in both training and validation sets, validation loss may become overly optimistic. A trajectory-level split gives a more meaningful validation result.

BC is simple and efficient because it does not require environment interaction during training. However, its weakness is also clear. The policy only learns from expert-visited states. During deployment, once the policy makes a small mistake, it may move the robot into a state that is not well represented in the training data. Then the next prediction becomes harder, and the error can accumulate over time. This is the covariate shift problem.

5 DAgger Method

DAgger is used to reduce the covariate shift problem of BC. Instead of training only on the original expert demonstrations, DAgger allows the current learner policy to interact with the environment and collect states that it actually visits. The expert then labels these learner-visited states. The newly labeled data are aggregated with the existing dataset, and the policy is retrained [1].

The DAgger pipeline in my implementation follows these steps:

1. Train an initial BC policy on the mixed demonstration dataset.
2. Roll out the current policy in the TwoArmLift environment.
3. Record the states visited by the learner policy.
4. Query the scripted expert to obtain expert actions for those states.
5. Add the new expert-labeled data to the training set.
6. Retrain the policy and repeat the process for several rounds.

A beta schedule is used during data collection. At each timestep, the action executed in the environment may come from either the expert or the student policy. The probability of using the expert action is controlled by β. Early rounds use more expert guidance, while later rounds expose the policy to more of its own induced state distribution.

A key implementation detail is that the executed action and the saved training label are separated. Even if the student action is executed in the environment, the supervised training label is still the expert action for that state. This matters because DAgger is not designed to imitate the student policy. It is designed to teach the learner what the expert would do in states reached by the learner.

Compared with BC, DAgger is more expensive because it requires rollouts and retraining. However, it is more suitable for closed-loop deployment because it directly trains on states that the learned policy is likely to visit.

6 Experimental Design

6.1 Evaluation Protocol

The final evaluation uses closed-loop rollouts in the robosuite environment. For each selected checkpoint, I evaluate 50 episodes. Each episode has a maximum horizon of 600 steps. If the robot successfully completes the lifting task, the episode is counted as successful. If the robot does not complete the task within the horizon, the episode is counted as a failure.

The main metrics are:

- **Success rate (SR):** the fraction of successful episodes;
- **Success count:** the number of successful episodes out of 50;
- **Mean episode steps:** the average number of steps before success or timeout.

Mean steps are interpreted together with success rate. A policy that often fails usually reaches the 600-step horizon, so a large mean step value often indicates poor closed-loop behavior. A good policy should have both high success rate and lower mean steps.

6.2 Checkpoint Selection

For BC, I evaluate the checkpoint selected by validation loss. For DAgger, I evaluate different rounds and select the best DAgger round for each noisy ratio based on final rollout performance.

The selection mainly considers success rate and mean episode steps, because they directly reflect whether the robot can complete the task.

The main experiments use a single random seed due to computational limits. This is a limitation, but the performance gaps under moderate noise are large enough to show clear trends. For example, at noisy ratio 0.5, BC has 0/50 successes while DAgger has 45/50 successes.

7 Main Results

Table 1 reports the final 50-episode evaluation results for BC and DAgger. The same results are shown in Figure 5 and Figure 6.

Table 1: Final 50-episode evaluation of BC and DAgger under selected noisy demonstration ratios.

Noisy ratio	BC SR	BC success	BC mean steps	Best DAgger round	DAgger SR	DAgger success	DAgger mean steps
0.0	0.72	36/50	265.64	2	0.44	22/50	404.08
0.3	0.18	9/50	531.06	3	0.68	34/50	334.52
0.5	0.00	0/50	600.00	3	0.90	45/50	204.08
1.0	0.00	0/50	600.00	3	0.24	12/50	536.42

In the clean setting, BC achieves a success rate of 0.72, while DAgger reaches 0.44. This result is expected because the clean demonstrations already provide a stable expert distribution. BC can directly fit this distribution and perform well. DAgger, by contrast, introduces additional learner-visited states during aggregation. These states may be harder than the original clean expert states, so DAgger does not automatically improve performance in the clean setting.

The situation changes when noisy demonstrations are introduced. At ratio 0.3, BC drops to 0.18 success rate. At ratio 0.5, BC completely fails with 0/50 successful episodes. This result shows that BC is highly sensitive to noisy action labels. Because BC directly fits the dataset actions, inconsistent demonstrations can distort the learned action mapping. In dual-arm lifting, this is especially harmful because both arms must coordinate timing, position, grasping, and lifting.

DAgger performs much better under moderate noise. At ratio 0.3, DAgger improves the success rate from 0.18 to 0.68. At ratio 0.5, the improvement is even stronger: BC fails completely, while DAgger achieves 0.90 success rate. This is the clearest result of my experiments. It shows that DAgger can reduce closed-loop distribution mismatch by adding expert-labeled learner-visited states.

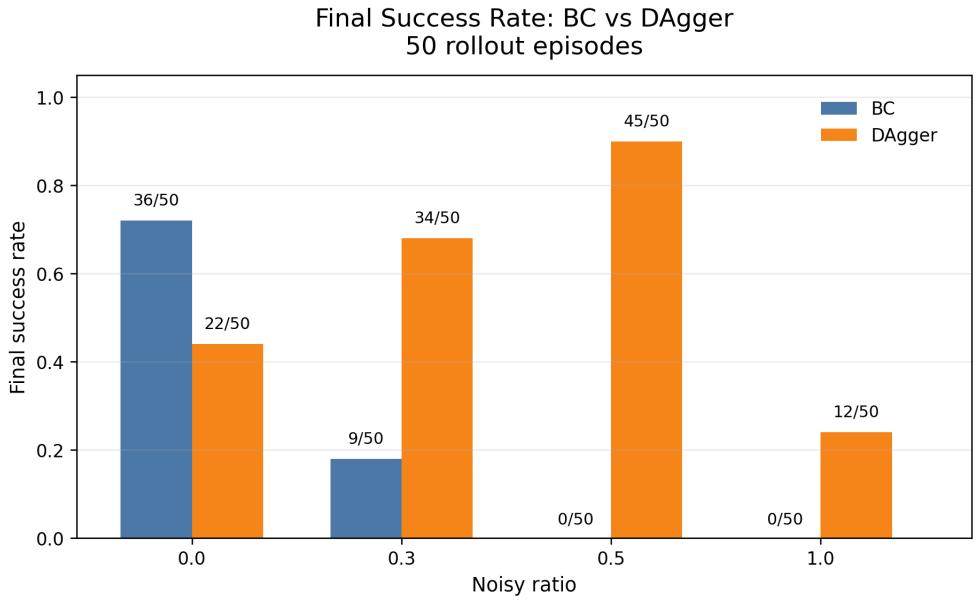

Figure 5: Final success rates of BC and DAgger under selected noisy demonstration ratios. BC performs well in the clean setting but quickly degrades as noisy demonstrations increase. DAgger substantially improves performance under moderate noise.

The mean-step results support the same conclusion. At ratio 0.5, BC reaches the maximum horizon of 600 steps, while DAgger completes episodes with a mean of 204.08 steps. At ratio 0.3, DAgger also reduces mean steps from 531.06 to 334.52. This means that DAgger not only succeeds more often, but also completes the task more efficiently when it succeeds.

However, the fully noisy ratio 1.0 setting remains difficult. BC again fails completely, while DAgger only achieves 12/50 successes. This suggests that DAgger can compensate for some covariate shift, but it cannot fully solve poor demonstration quality. If the base dataset is dominated by noisy and inconsistent behaviors, expert-labeled aggregation data may still be insufficient.

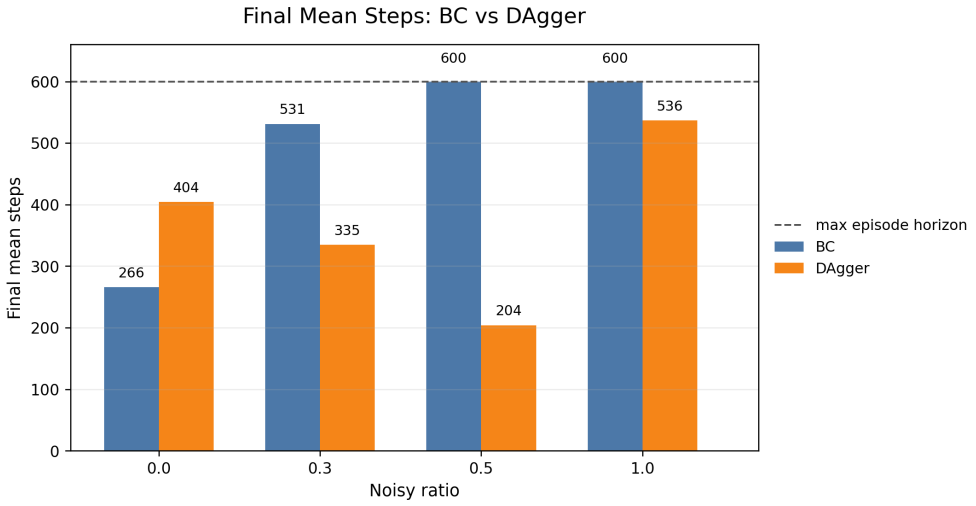

Figure 6: Final mean episode steps of BC and DAgger under selected noisy demonstration ratios. Failed episodes usually reach the 600-step horizon, while successful DAgger policies complete the task faster under moderate noise.

Table 2 reports the best DAgger round for each ratio. Except for the clean setting, the best checkpoint usually appears after three aggregation rounds. This supports the idea that iterative exposure to learner-visited states is useful under noisy demonstrations.

Table 2: Best DAgger round selected for each noisy ratio based on final rollout performance.

Noisy ratio	**Best DAgger round**	**DAgger SR**	**DAgger success**	**DAgger mean steps**
0.0	2	0.44	22/50	404.08
0.3	3	0.68	34/50	334.52
0.5	3	0.90	45/50	204.08
1.0	3	0.24	12/50	536.42

8 Covariate Shift Analysis

The results can be explained by covariate shift and compounding errors. BC is trained on expert demonstration states, but at test time the policy executes its own predicted actions. If the prediction is slightly wrong, the robot may enter a state that the expert demonstrations did not cover well. The next prediction then becomes harder, and errors can accumulate over the episode.

This problem is more serious in dual-arm manipulation than in simple tasks. The two arms must coordinate through the whole episode. A small timing mismatch can make the grasp unstable. One arm may close too early, or one arm may lift before the other arm has secured the object. These errors may look small at the action level but can cause complete task failure.

DAgger reduces this problem because it trains on states visited by the learner. These states are relabeled by the expert, so the model learns what the expert would do when the student reaches imperfect states. This gives the policy recovery examples. The strong improvement at ratios 0.3 and 0.5 shows that this mechanism is effective when the demonstrations are noisy but still contain enough task structure.

This also explains why training loss alone is not enough. A policy may have a reasonable one-step prediction loss but still fail in closed-loop rollout. In this project, the most important evaluation is not only whether the model fits the dataset, but whether it can complete the task when its own actions are repeatedly executed in the environment.

9 Failure Cases and Limitations

Although DAgger improves robustness under moderate noise, it does not solve all problems. The fully noisy ratio 1.0 setting remains difficult. DAgger achieves 12/50 successes, which is better than BC, but it is still not reliable. This means that expert relabeling can help, but it cannot fully repair a base dataset whose action distribution is too noisy.

The ratio 0.8 diagnostic case also shows an unstable high-noise failure. In that setting, both BC and DAgger fail completely. A possible reason is that the ratio 0.8 dataset is dominated by noisy demonstrations but still contains a small amount of clean data. This may create an inconsistent mixed distribution. The policy may learn to approach the handles but fail to reliably switch to the close-and-lift stage.

Another limitation is the use of a single random seed. A more complete evaluation would include multiple seeds and confidence intervals. However, the main performance gaps are large enough to support the conclusions. The difference at ratio 0.5 is especially clear: BC has 0/50 successes, while DAgger has 45/50 successes.

Finally, both BC and DAgger use an MLP policy. This is suitable for a classical baseline with low-dimensional states, but it may not be expressive enough for all dual-arm behaviors. Dual-arm manipulation may contain multi-modal actions, and an MLP trained with MSE may average across different valid behaviors. This limitation connects naturally to the Diffusion Policy part of the project.

10 Connection to Student 1 and Student 3 Work

Student 1's work provides the data foundation for my experiments. Without the environment, scripted expert, and noisy demonstration generation pipeline, my BC and DAgger comparison would not be possible. The clean and noisy datasets allow me to test how traditional imitation learning methods behave under different data quality conditions.

Student 3's work explores Diffusion Policy, which is more advanced than BC and DAgger. Diffusion Policy represents visuomotor policies through a conditional denoising diffusion process and is designed to model complex action distributions [3]. My results help motivate that part of the project. Since BC is sensitive to noisy demonstrations and DAgger still has limitations under high noise, a stronger policy model is useful. Diffusion Policy can model action generation more flexibly and can use action chunking to improve temporal consistency.

Therefore, my work plays a middle role in the integrated project. Student 1 builds the data and environment foundation. My part tests classical imitation learning baselines and identifies their limitations. Student 3 then explores a more expressive policy-learning method.

11 Conclusion

This report presents my contribution to the integrated dual-arm imitation learning project. I implemented and evaluated Behavior Cloning and DAgger on mixed clean/noisy demonstration datasets. The final evaluation uses 50 closed-loop rollout episodes and reports success rate, success count, and mean episode steps.

The main findings are clear. BC performs well when demonstrations are clean, but it quickly breaks down when noisy demonstrations increase. DAgger significantly improves performance under moderate noise by adding expert-labeled learner-visited states. The strongest result appears at noisy ratio 0.5, where BC has 0/50 successes and DAgger reaches 45/50 successes. However, very high-noise settings remain difficult, showing that DAgger is helpful but not universally robust.

Overall, my part shows that closed-loop evaluation is essential in imitation learning. A policy should not be judged only by supervised loss. It must be tested in the environment, where its own actions affect future states. The BC and DAgger results reveal the importance of covariate shift, data quality, and recovery behavior in long-horizon dual-arm manipulation.

Student 2 Appendix: Additional Diagnostics

This appendix reports additional diagnostic experiments for my BC and DAgger component. These results are not used as the main evidence because the main conclusions are based on the final 50-episode closed-loop evaluation. Instead, the appendix provides supporting evidence for data-size sensitivity, high-noise failure cases, and BC training behavior.

Supplementary BC-only Data-Size Ablation

To further understand BC, I conducted a supplementary BC-only ablation with different numbers of demonstrations. The goal is to check whether adding more demonstrations consistently improves BC performance under different noisy-demonstration ratios.

Table A1 shows the 50-episode evaluation results. The results are not strictly monotonic, especially under noisy settings. For clean demonstrations, increasing the number of demonstrations generally improves BC from 0.20 at $N = 20$ to 0.74 at $N = 80$. However, under noisy demonstrations, more data does not always lead to better performance. For example, under ratio 0.2, the success rate decreases from 0.76 at $N = 40$ to 0.26 at $N = 80$. Under ratio 0.3, the success rate is 0.00 at $N = 40$ but increases to 0.60 at $N = 80$.

These results should not be interpreted as a claim that dataset size has a simple monotonic effect. Instead, they suggest that for BC, the quality and composition of the selected demonstrations can matter as much as the number of demonstrations. This is especially true when the dataset contains noisy action labels.

Table A1: Supplementary BC-only data-size ablation under different noisy-demonstration ratios.

Noisy ratio	N	**Clean demos**	**Noisy demos**	**BC success**	**BC SR**
0.0	20	20	0	10/50	0.20
0.0	40	40	0	20/50	0.40
0.0	80	80	0	37/50	0.74
0.2	20	16	4	26/50	0.52
0.2	40	32	8	38/50	0.76
0.2	80	64	16	13/50	0.26
0.3	20	14	6	9/50	0.18
0.3	40	28	12	0/50	0.00
0.3	80	56	24	30/50	0.60

The main takeaway is that demonstration count alone is not sufficient to explain BC performance. In noisy imitation learning, data quality and subset composition can be as important as dataset size.

High-Noise Diagnostic Case at Ratio 0.8

In addition to the main ratios, I evaluated ratio 0.8 as a high-noise diagnostic case. This result was excluded from the main trend figure because it behaves more like a failure case than a smooth intermediate point.

Table A2: High-noise diagnostic result at noisy ratio 0.8.

Noisy ratio	Method	Best round	Success	SR	Mean steps	Interpretation
0.8	BC	–	0/50	0.00	600.00	Failed under high-noise mixed data
0.8	DAgger	1	0/50	0.00	600.00	Unstable high-noise failure case

Both BC and DAgger failed completely at ratio 0.8. A possible explanation is that this mixed dataset is dominated by noisy demonstrations while still retaining a small clean subset, creating an inconsistent training distribution. Because this result is unstable and not central to the main trend, it is reported only as a diagnostic case.

BC Training Curves for Representative Noise Ratios

Figure A1 shows BC training curves for three representative noise ratios: noise0, noise30, and noise50. This figure is only a training diagnostic. It shows that clean data produces much lower MSE loss, while noisy data keeps the supervised loss at a higher level.

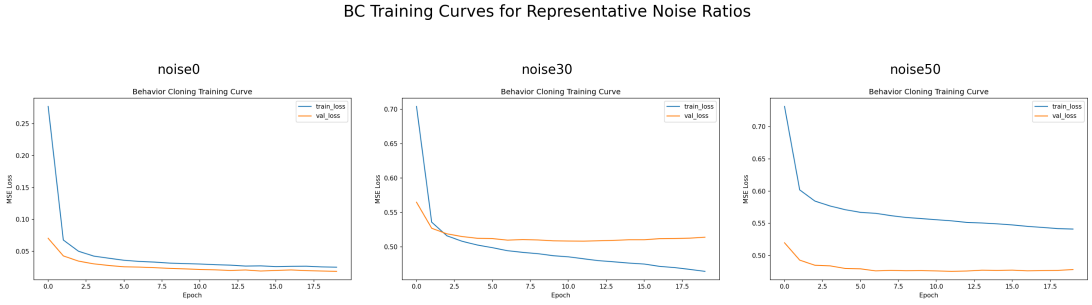

Figure A1: BC training curves for representative noisy demonstration ratios. Clean data converges to a much lower MSE loss, while noisy demonstrations keep the supervised loss at a higher level.

This supports the observation that BC is sensitive to noisy action labels. However, the main evaluation remains closed-loop rollout success rate and mean episode steps. Therefore, the training curves are used only as supporting evidence, not as the main performance result.

Summary of Supplementary Findings

The supplementary results support the main report in three ways. First, the BC-only data-size ablation shows that more demonstrations do not always improve BC under noisy settings. Second, the ratio 0.8 diagnostic case shows that high-noise mixed demonstrations can cause complete failure even for DAgger. Third, the BC training curves show that noisy data makes supervised fitting harder. Together, these findings support the conclusion that closed-loop imitation learning performance depends on data quality, data distribution, and policy-induced state distribution.

References

[1] Ross, S., Gordon, G. J., & Bagnell, J. A. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics*.

[2] Zhu, Y., Wong, J., Mandlekar, A., Martín-Martín, R., Joshi, A., Lin, K., Maddukuri, A., Nasiriany, S., & Zhu, Y. (2020). robosuite: A modular simulation framework and benchmark for robot learning. *arXiv preprint arXiv:2009.12293*.

[3] Chi, C., Xu, Z., Feng, S., Cousineau, E., Du, Y., Burchfiel, B., Tedrake, R., & Song, S. (2023). Diffusion Policy: Visuomotor policy learning via action diffusion. *arXiv preprint arXiv:2303.04137*.

GitHub Repository

The project code and related materials are available at:

https://github.com/MillenRosen/diffusion-policy-dual-arm